Programming Project Report

Warren Roberts
100450116

Problem Statement:

The goal of this assignment was to design, implement, and learn how terrain generation works using OpenGL. This took a lot of work and offered design and architectural challenges. To do this I implemented a 100 x 100 grid, applied mathematical height functions and Perlin noise to create pseudo-random generation. Making the terrain look natural took a while to achieve but was beautiful in the end! Normal inputs included keyboard commands during runtime such as "WASD" and Q or E to move and transform the scale. Also, I started testing the program with simple functions that utilize sin, cos, and e to generate a base surface. As the project progressed, I started to implement Perlin noise for structured pseudo-random terrain variation. The program outputs a 3D render of a desert-themed mountain, allowing you to traverse and scale for ease of viewing. Error handling mainly involved preventing invalid index access when processing terrain neighbors during smoothing and rendering. Because terrain generation uses neighboring grid points frequently, boundary conditions had to be handled carefully so that edge cells did not attempt to access values outside the valid terrain grid.

Design:

The project was designed with MVC architecture in mind, and I created many directories to uphold that idea. For instance, I had terrain, view, controller, core, and app directories to separate ideas and maintain MVC structure. This helped with organization, searching for bugs, and general peace of mind. The main data structures used were vectors to contain X, Y, and Z coordinates that will be accessed to draw the terrain. This allowed for efficient indexing after making a getIndex helper method. I also created structs for RGB, Vec3, and Vec2 to be used for coloring and drawing. For terrain generation I used Perlin noise and watched a very handy video that demonstrated its implementation in C++. With this algorithm I was able to create quality random noise. Linear interpolation was necessary for this implementation. A link to the video will be below. For smoothing I used local 3x3 averaging, i.e. local neighbor averaging was used to reduce spikes and keep slopes realistic. A major design advantage was the generation techniques could be changed quickly and efficiently. For example, color mapping logic can be adjusted without changing the terrain generation formula. A con would be that some terrain features required extensive tuning between files to obtain a realistic outcome.

Implementation:

I began by creating empty files and directories of everything I thought I would need, then added a notes section at the top of each file to support the organization and make it easy to identify the files' purpose. After this I created main.cpp and app.h /

app.cpp. These files were the backbone of the system, allowing for every component to be combined in an organized and simple manner. There was no starter code, so I began working extremely early, only a couple days after posting. To keep on track, I worked almost every day from then to now on refining my code and implementing new features. I loved this project and wanted to do even more than I did do, but time cut me short. I implemented a user interaction system via controller to allow the user to transform on the plane, allowing them to move camera positions and zoom in / out with glOrtho. I kept extending this code everyday picking a job to do and trying my best to make it perfect. In the last day or two I spent a lot of time trying make a typical mountain to be what is seen when you launch the app (which is currently a pretty cool looking desert-themed mountain if I do say so myself).

Video for Perlin implementation:

- https://www.youtube.com/watch?v=kCleHqb60Cw

ChatGPT Prompts:

• Interaction 1

• Prompt

- I would like to zoom out my camera in OpenGL. I read `glScale` is good, but I get an error saying identifier `"glScale"` is undefined.

• AI Assistance Summary

- The AI explained that `glScale` is not a valid OpenGL function and that the correct fixed-pipeline functions are `glScalef` and `glScaled`. It also explained the difference between scaling the scene and moving the camera backward using translation, noting that scene scaling can be acceptable for simple projects while camera movement is the more general long-term solution.

• How it was used

- This guidance helped clarify why the function name was failing and supported the decision to use `glScalef` for simple zoom control in the terrain viewer.
- ---

• Interaction 2

• Prompt

- I asked whether `glScalef` was appropriate for keyboard-controlled zooming where one key zooms out by 10% and another zooms in by 10%.

• AI Assistance Summary

- The AI explained that `glScalef` is acceptable for a simple terrain zoom system at the current homework stage, especially when using legacy OpenGL. It also noted that repeated scaling should be controlled carefully to avoid compounding distortion.

- ## How it was used
- This confirmed that scene scaling was a reasonable temporary zoom approach for Homework 3.
- ---

• Interaction 3

• Prompt

- I asked how terrain smoothing works, specifically 3×3 and 5×5 local averaging, without requesting code.

• AI Assistance Summary

- The AI explained that local averaging replaces each height value with the average of neighboring values. For 3×3 smoothing, nine surrounding values are averaged; for 5×5 smoothing, twenty-five surrounding values are averaged. It also explained why smoothing reduces sharp spikes, why temporary storage is needed to avoid overwriting neighbor calculations during a pass, and how repeated smoothing affects terrain realism.

• How it was used

- This helped guide the smoothing design in the terrain generation pipeline after noise was added.
- ---

• Interaction 4

• Prompt

- I asked what the correct OpenGL call is for registering an idle function.

• AI Assistance Summary

- The AI explained that GLUT uses `glutIdleFunc(...)` to register a function that runs when no other events are occurring. It also explained the difference between update logic in idle callbacks and rendering logic in display callbacks.

• How it was used

- This clarified callback registration during the OpenGL application setup.
- ---

• Interaction 5

• Prompt

- I asked how to design an elevation-based terrain color map using mountain colors from low elevation to snow peaks while keeping MVC architecture clean.

• AI Assistance Summary

- The AI explained how to map terrain height ranges into normalized values and assign elevation thresholds to terrain colors. It recommended separating terrain color logic into its own helper module (such as a `TerrainColorMap`) while

keeping terrain height data in the model and rendering color application inside the view.

- **How it was used**
- This influenced the decision to separate terrain coloring logic from rendering logic and keep color mapping modular.
- ---

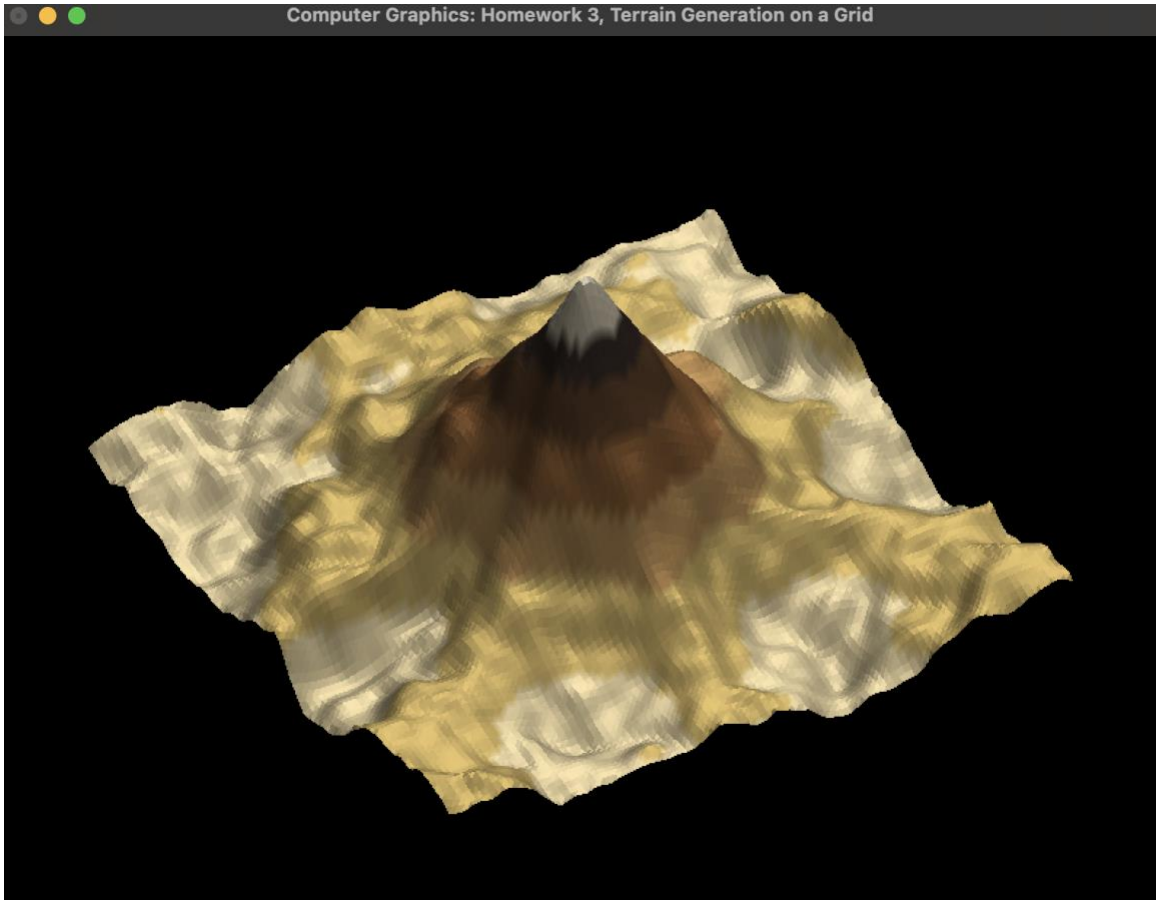
- **Interaction 6**
- **Prompt**
- I asked whether my wireframe terrain rendering method was correctly applying elevation-based colors per vertex.
- **AI Assistance Summary**
- The AI explained that in line rendering, each endpoint vertex needs its own color assignment because OpenGL interpolates color between vertices. It also noted that color should be recomputed for each connected neighbor rather than only once per starting point.
- **How it was used**
- This improved the color behavior across wireframe terrain edges.
- ---

- **Interaction 7**
- **Prompt**
- I requested multiple mountain-themed 7-color palettes in float RGB format, including desert and forest themes.
- **AI Assistance Summary**
- The AI generated several themed elevation palettes including desert, forest, volcanic, alpine, and fantasy mountain styles, each arranged from low elevation to high elevation.
- **How it was used**
- These palettes were used to experiment with different terrain appearances and choose visually appropriate elevation bands.

Testing:

Testing was a continuous journey, with each step being tested before moving onto the next. Normal inputs were tested such as movement and zooming, as well as implementing wireframe rendering. This wireframe served me very well and I even made it, so the grid was colored based on the height. This allowed me to view changes made to the terrain, noise, smoothing, and coloring with ease. After this I developed a triangle-based rendering system to create a real filled terrain, and then lighting was added to bring more realism. I tested this with common light settings and tweaked it for over an hour till

I found something that looked quality to me. Render stability was tested well with scaling and movement as it had to continuously redraw from a different point of view. Everything worked as expected in the end and I am very happy with the results.



Conclusions:

Overall, this was an awesome project and opportunity to learn and implement awesome ideas and algorithms. I think it's amazing how pseudo-random noise generation can really bring a clump of polygons to a realistic mountain. Before this project I did not understand the need for engineered randomness, but now I understand its usefulness and will try to apply it everywhere I see it applicable. This project was a massive success in my books, and even though I don't have many hills I have one large that looks like a view from above (like from an airplane as requested). I am afraid it may not be exactly what this assignment described as a result, but this is what I thought looked the best in the end. I would love to use this terrain generation to create a large world for a game that users can interact with, traverse, and collide with. This project took about two weeks to complete and if I were to do it again, I would consider adding more lighting features to fully implement Phong lighting and add mouse-drag based rotations for the camera to give a truly immersive viewer experience!